

Hands-On Module 1 — Exploratory Data Analysis Dataset Spotify

Program: GDGOC ITB — Artificial Intelligence (Module 1) **Nama:** (isi nama kamu) | **NIM:** (isi NIM)
Dataset: Spotify Songs — TidyTuesday 2020-01-21

Notebook ini melakukan *exploratory data analysis* (EDA) menyeluruh atas dataset lagu Spotify (~33 ribu lagu, 6 genre playlist) menggunakan **Pandas** dan **NumPy**. Tujuannya bukan sekadar menjalankan fungsi, tapi mengambil keputusan analitis yang bisa dipertanggungjawabkan dan menarik insight yang bermakna.

Struktur notebook mengikuti tahapan tugas: **Stage 1** (setup & inspeksi), **Stage 2** (genre & popularitas), **Stage 3** (analisis NumPy), **Stage 4** (dokumentasi & refleksi AI tool), dan **Bonus 1-3**. Setiap keputusan dijelaskan pada *markdown cell* tepat sebelum atau sesudah kodenya, supaya alur penalarannya jelas.

Stage 1 — Setup dan Inspeksi Awal

Sebelum analisis apa pun, saya perlu mengenali datanya dulu: seberapa besar, tipe kolomnya apa, ada yang kosong atau duplikat tidak, dan bagaimana sebaran nilainya. Tahap ini menentukan keputusan pembersihan data yang dipakai di seluruh notebook.

Soal 1 — Memuat dataset

Dataset dimuat langsung dari URL mentah GitHub menggunakan `pd.read_csv()`, tanpa perlu mengunduh manual. `df.head()` saya panggil untuk melihat lima baris pertama dan mengenali bentuk datanya.

```
In [1]: import pandas as pd
import numpy as np

url = "https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2020/2020-01-21/spotify_songs.csv"
df = pd.read_csv(url)
df.head()
```

	track_id	track_name	track_artist	track_popularity	track_album_id	track
0	6f807x0ima9a1j3VPbc7VN	I Don't Care (with Justin Bieber) - Loud Luxur...	Ed Sheeran	66	2oCs0DGTsRO98Gh5ZSI2Cx	I [Justi
1	0r7CVbZTWZgbTCYdfa2P31	Memories - Dillon Francis Remix	Maroon 5	67	63rPSO264uRjW1X5E6cWv6	M
2	1z1Hg7Vb0AhHDIEmnDE79I	All the Time - Don Diablo Remix	Zara Larsson	70	1HoSmj2eLcsrR0vE9gThr4	Al
3	75FpbthrwQmzHIBJLuGdC7	Call You Mine - Keanu Silva Remix	The Chainsmokers	60	1nqYsOef1yKKuGOVchbsk6	Call
4	1e8PAfcKUYoKkxPhrHqw4x	Someone You Loved - Future Humans Remix	Lewis Capaldi	69	7m7vv9wIQ4i0LFuJiE2zsQ	Some (

5 rows × 7 columns

Soal 2 — Inspeksi awal: shape, dtypes, dan missing value

Saya periksa dimensi data (`shape`), tipe tiap kolom (`info`), lalu menghitung jumlah dan proporsi nilai kosong per kolom.

```
In [2]: print("Shape:", df.shape)
df.info()
```

```
Shape: (32833, 23)
<class 'pandas.DataFrame'>
RangeIndex: 32833 entries, 0 to 32832
Data columns (total 23 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   track_id                             32833 non-null  str
1   track_name                           32828 non-null  str
2   track_artist                         32828 non-null  str
3   track_popularity                     32833 non-null  int64
4   track_album_id                      32833 non-null  str
5   track_album_name                    32828 non-null  str
6   track_album_release_date            32833 non-null  str
7   playlist_name                       32833 non-null  str
8   playlist_id                         32833 non-null  str
9   playlist_genre                      32833 non-null  str
10  playlist_subgenre                   32833 non-null  str
11  danceability                        32833 non-null  float64
12  energy                             32833 non-null  float64
13  key                                 32833 non-null  int64
14  loudness                           32833 non-null  float64
15  mode                               32833 non-null  int64
16  speechiness                        32833 non-null  float64
17  acousticness                       32833 non-null  float64
18  instrumentalness                   32833 non-null  float64
19  liveness                           32833 non-null  float64
20  valence                            32833 non-null  float64
21  tempo                              32833 non-null  float64
22  duration_ms                        32833 non-null  int64
dtypes: float64(9), int64(4), str(10)
memory usage: 5.8 MB
```

```
In [3]: missing = pd.DataFrame({
    "missing_count": df.isnull().sum(),
    "missing_pct": (df.isnull().mean() * 100).round(3)
})
missing[missing["missing_count"] > 0]
```

```
Out[3]:
```

	missing_count	missing_pct
track_name	5	0.015
track_artist	5	0.015
track_album_name	5	0.015

Pembahasan. Dataset berisi **32.833 baris x 23 kolom**. Dari pemeriksaan di atas, hanya tiga kolom yang punya nilai kosong — `track_name`, `track_artist`, dan `track_album_name` — masing-masing **5 baris** saja (sekitar 0,015% dari total).

Instruksi meminta: kolom dengan missing > 20% harus dibuang. Pada dataset ini **tidak ada satu pun kolom yang mendekati ambang itu**, sehingga **tidak ada kolom yang saya buang** — membuang kolom dengan missing sekecil ini justru akan membuang informasi yang valid. Yang saya lakukan hanya membuang **5 baris** yang `track_name` / `track_artist` -nya kosong, karena baris tanpa identitas lagu tidak berguna untuk analisis dan jumlahnya tidak signifikan.

```
In [4]: df = df.dropna(subset=["track_name", "track_artist"]).reset_index(drop=True)
```

Soal 3 — Menangani duplikat

Saya membandingkan tiga angka: duplikat baris penuh, duplikat berdasarkan `track_id`, dan jumlah `track_id` unik.

```
In [5]: print("Duplikat baris full:", df.duplicated().sum())
print("Duplikat track_id:", df.duplicated(subset=["track_id"]).sum())
print("Track_id unik:", df["track_id"].nunique())
```

Duplikat baris full: 0
Duplikat track_id: 4476
Track_id unik: 28352

Menarik: **tidak ada duplikat baris penuh (0)**, tetapi ada **4.477 track_id yang berulang**. Untuk memahami penyebabnya, saya lihat satu contoh lagu yang track_id -nya paling sering muncul.

```
In [6]: df[df["track_id"] == df["track_id"].value_counts().index[0]][
        ["track_name", "track_artist", "playlist_genre", "playlist_subgenre"]
        ]
```

```
Out[6]:
```

	track_name	track_artist	playlist_genre	playlist_subgenre
133	Closer (feat. Halsey)	The Chainsmokers	pop	dance pop
1731	Closer (feat. Halsey)	The Chainsmokers	pop	post-teen pop
2478	Closer (feat. Halsey)	The Chainsmokers	pop	electropop
4535	Closer (feat. Halsey)	The Chainsmokers	pop	indie pop
7369	Closer (feat. Halsey)	The Chainsmokers	rap	southern hip hop
17464	Closer (feat. Halsey)	The Chainsmokers	latin	tropical
18355	Closer (feat. Halsey)	The Chainsmokers	latin	latin pop
19766	Closer (feat. Halsey)	The Chainsmokers	latin	latin hip hop
23779	Closer (feat. Halsey)	The Chainsmokers	r&b	hip pop
30624	Closer (feat. Halsey)	The Chainsmokers	edm	pop edm

```
In [7]: df_unique = df.drop_duplicates(subset="track_id", keep="first").reset_index(drop=True)
print("df (asli) :", df.shape)
print("df_unique :", df_unique.shape)
```

```
df (asli) : (32828, 23)
df_unique : (28352, 23)
```

Strategi deduplikasi & justifikasi. Dari contoh di atas terlihat bahwa duplikat track_id **bukan error data**, melainkan satu lagu yang sama yang masuk ke beberapa playlist sekaligus — dan tiap playlist punya genre/subgenre berbeda. Misalnya "Closer" (The Chainsmokers) muncul di pop, rap, latin, r&b, dan edm.

Karena itu saya **tidak** membuang duplikat secara membabi buta, melainkan menyiapkan **dua versi data**:

- **df (asli)** — dipakai untuk analisis berbasis **genre** (Stage 2). Di sini keanggotaan multi-genre adalah sinyal yang valid dan justru ingin dipelajari; kalau dibuang, representasi genre malah rusak.
- **df_unique** — hasil dedup berdasarkan track_id (kolom yang dipilih karena merupakan identitas unik tiap rekaman). Dipakai untuk analisis **fitur audio per-lagu** (Stage 3 & bonus), supaya satu lagu tidak terhitung berkali-kali dan statistiknya tidak bias ke lagu yang kebetulan ada di banyak playlist.

Prinsipnya: **pemilihan versi data mengikuti unit analisis** dari pertanyaan yang sedang dijawab.

Soal 4 — Tabel ringkasan statistik (mean, median, std, IQR manual)

Untuk setiap kolom numerik saya hitung mean, median, standar deviasi, dan IQR. IQR dihitung **manual** dengan NumPy (np.nanpercentile), bukan scipy, sesuai instruksi.

```
In [8]: numeric_cols = df.select_dtypes(include=np.number).columns
summary = pd.DataFrame({
    "mean": df[numeric_cols].mean(),
    "median": df[numeric_cols].median(),
    "std": df[numeric_cols].std(),
})

q1 = np.nanpercentile(df[numeric_cols].values, 25, axis=0)
q3 = np.nanpercentile(df[numeric_cols].values, 75, axis=0)
summary["IQR"] = q3 - q1

summary.round(3)
```

Out[8]:

	mean	median	std	IQR
track_popularity	42.484	45.000	24.980	38.000
danceability	0.655	0.672	0.145	0.198
energy	0.699	0.721	0.181	0.259
key	5.374	6.000	3.612	7.000
loudness	-6.720	-6.166	2.989	3.526
mode	0.566	1.000	0.496	1.000
speechiness	0.107	0.062	0.101	0.091
acousticness	0.175	0.080	0.220	0.240
instrumentalness	0.085	0.000	0.224	0.005
liveness	0.190	0.127	0.154	0.155
valence	0.511	0.512	0.233	0.362
tempo	120.884	121.984	26.904	33.957
duration_ms	225796.830	216000.000	59836.492	65776.750

Pembahasan. IQR (Interquartile Range = $Q3 - Q1$) menggambarkan rentang **50% data di tengah**, sehingga lebih **tahan terhadap outlier** dibanding standar deviasi yang mudah tertarik oleh nilai ekstrem. Itulah alasan tugas meminta IQR dihitung manual — agar terlihat bahwa ia sekadar "selisih persentil ke-75 dan ke-25", bukan sesuatu yang ajaib. Beberapa pengamatan: pada `track_popularity`, selisih antara mean dan median menandakan distribusinya tidak simetris, sedangkan `duration_ms` memiliki IQR besar yang wajar karena durasi lagu memang sangat beragam.

Stage 2 — Analisis Genre dan Popularitas

Pada tahap ini saya memakai `df (asli)`, karena unit analisisnya adalah kemunculan lagu pada genre playlist tertentu — persis sesuai keputusan deduplikasi di Stage 1.

Soal 5 — Distribusi statistik per genre

Dengan `groupby`, saya hitung mean, std, dan median dari `track_popularity`, `danceability`, `energy`, dan `valence` untuk tiap `playlist_genre`.

```
In [9]: genre_stats = df.groupby("playlist_genre")[
        ["track_popularity", "danceability", "energy", "valence"]
        ].agg(["mean", "std", "median"])
        genre_stats.round(3)
```

Out[9]:

	track_popularity			danceability			energy			valence		
	mean	std	median	mean	std	median	mean	std	median	mean	std	median
playlist_genre												
edm	34.834	23.154	36.0	0.655	0.124	0.659	0.802	0.139	0.830	0.401	0.226	0.37
latin	47.045	25.413	50.0	0.713	0.115	0.729	0.708	0.152	0.729	0.605	0.222	0.62
pop	47.745	25.158	52.0	0.639	0.128	0.652	0.701	0.171	0.727	0.504	0.220	0.50
r&b	41.224	25.895	44.0	0.670	0.138	0.689	0.591	0.179	0.596	0.531	0.226	0.54
rap	43.238	23.287	47.0	0.718	0.136	0.737	0.651	0.170	0.665	0.505	0.225	0.51
rock	41.728	24.825	46.0	0.521	0.140	0.523	0.733	0.195	0.775	0.537	0.229	0.53

Pengamatan. Tabel ini memberi "potret" tiap genre. Beberapa pola langsung terlihat: **pop** cenderung punya popularitas rata-rata tertinggi, sementara **edm** justru paling *danceable* dan energik tetapi popularitasnya paling rendah. Angka-angka inilah yang menjadi pijakan untuk pertanyaan berikutnya.

Soal 6 — Genre dengan variance popularitas tertinggi

Variance mengukur seberapa "tersebar" nilai popularitas dalam satu genre.

```
In [10]: var_pop = df.groupby("playlist_genre")["track_popularity"].var().sort_values(ascending=False)
var_pop.round(2)
```

```
Out[10]: playlist_genre
r&b      670.53
latin    645.80
pop       632.94
rock      616.29
rap       542.29
edm       536.12
Name: track_popularity, dtype: float64
```

Jawaban & interpretasi bisnis. Genre dengan variance popularitas tertinggi adalah **r&b (≈670,5)**, disusul latin dan pop; yang terendah adalah edm.

Dari sudut pandang bisnis rekomendasi konten, variance tinggi berarti popularitas lagu-lagu r&b **sangat tidak seragam** — ada yang menjadi *mega-hit*, tapi banyak pula yang sepi peminat. Untuk sistem rekomendasi, genre seperti ini lebih **sulit diprediksi**: platform tidak bisa berasumsi "karena ini r&b, pasti disukai". Konsekuensinya, genre *high-variance* menuntut **personalisasi yang lebih dalam** (per-pengguna), bukan rekomendasi rata-rata. Sebaliknya, edm dengan variance rendah lebih "aman" diprediksi karena tingkat popularitas lagunya relatif konsisten.

Soal 7 — Sepuluh artis dengan lagu terbanyak & popularitasnya

Saya memakai **df_unique** di sini agar yang dihitung adalah **lagu unik per artis**, bukan kemunculan di banyak playlist (yang akan membuat artis tertentu tampak lebih produktif dari kenyataannya).

```
In [11]: top10 = df_unique["track_artist"].value_counts().head(10)
print(top10)

pop_top10 = (df_unique[df_unique["track_artist"].isin(top10.index)]
              .groupby("track_artist")["track_popularity"].mean()
              .sort_values(ascending=False))
print(pop_top10.round(2))
```

```
track_artist
Queen                      130
Martin Garrix              87
Don Omar                  84
David Guetta              81
Dimitri Vegas & Like Mike  68
Hardwell                  68
Drake                     68
The Chainsmokers           66
Logic                     65
Guns N' Roses             63
Name: count, dtype: int64
track_artist
David Guetta              49.37
The Chainsmokers           49.23
Queen                    42.40
Logic                    41.91
Drake                    41.81
Martin Garrix            41.40
Don Omar                 39.06
Hardwell                 36.32
Dimitri Vegas & Like Mike 36.22
Guns N' Roses            27.21
Name: track_popularity, dtype: float64
```

Jawaban: apakah volume berkorelasi dengan kualitas? Artis dengan lagu terbanyak adalah **Queen (130 lagu unik)**, diikuti Martin Garrix dan Don Omar. Namun di antara sepuluh artis tersebut, yang **mean popularitasnya tertinggi adalah David Guetta (≈49,4)** — bukan Queen (≈42,4).

Jadi **volume tidak berkorelasi dengan kualitas** pada dataset ini. Queen paling produktif tetapi popularitas rata-ratanya hanya peringkat ketiga, dan Guns N' Roses bahkan punya banyak lagu namun popularitas rata-ratanya paling rendah (≈27,2). Banyaknya jumlah lagu tidak menjamin tiap lagu populer.

Soal 8 — Filter multi-kondisi

Saya menyaring lagu yang memenuhi **semua** syarat sekaligus: `track_popularity > 70`, `danceability > 0.7`, `energy > 0.6`, dan `duration_ms < 240000`. Tiap kondisi dibungkus tanda kurung dan disambung dengan `&` (operator elemen-wise Pandas; `and` / `or` biasa tidak berlaku untuk Series).

```
In [12]: mask = (df["track_popularity"] > 70) & (df["danceability"] > 0.7) & \
           (df["energy"] > 0.6) & (df["duration_ms"] < 240000)
           filtered = df[mask]

           print("Jumlah lolos (df, per playlist-entry):", len(filtered))
           print(filtered["playlist_genre"].value_counts())

           # Pembanding pada lagu unik
           mask_u = (df_unique["track_popularity"] > 70) & (df_unique["danceability"] > 0.7) & \
                    (df_unique["energy"] > 0.6) & (df_unique["duration_ms"] < 240000)
           print("\nJumlah lolos (df_unique, lagu unik):", df_unique[mask_u].shape[0])
```

Jumlah lolos (df, per playlist-entry): 1112

playlist_genre

latin 396

pop 264

rap 173

r&b 135

edm 111

rock 33

Name: count, dtype: int64

Jumlah lolos (df_unique, lagu unik): 579

Jawaban. Sebanyak **1.112 baris lolos** pada `df`, dengan **genre latin paling dominan (396)**, disusul pop. Jika dihitung pada lagu unik (`df_unique`), jumlahnya **579 lagu**.

Saya menampilkan kedua angka agar efek duplikat antar-playlist tetap transparan. Versi `df` saya pakai untuk melihat dominasi genre, karena sebuah lagu memang sah dihitung pada setiap genre playlist yang memuatnya. Insight-nya: profil "**populer + sangat danceable + energik + durasi pendek**" paling banyak ditemukan di **latin dan pop** — masuk akal, sebab itu ciri khas lagu *dance/party*.

Stage 3 — Analisis NumPy

Seluruh tahap ini memakai `df_unique` (analisis fitur audio per-lagu) dan menekankan operasi **vektorisasi** — tanpa loop Python dan tanpa sklearn.

Soal 9 — Ekstraksi fitur & normalisasi min-max (vektorisasi)

Sembilan fitur audio diekstrak menjadi array NumPy, lalu dinormalisasi ke rentang [0, 1].

```
In [13]: feats = ["danceability", "energy", "loudness", "speechiness",
                 "acousticness", "instrumentalness", "liveness", "valence", "tempo"]
           X = df_unique[feats].values
           print("Shape:", X.shape)

           X_norm = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))

           print("Min tiap kolom:", X_norm.min(axis=0).round(2))
           print("Max tiap kolom:", X_norm.max(axis=0).round(2))
```

Shape: (28352, 9)

Min tiap kolom: [0. 0. 0. 0. 0. 0. 0. 0. 0.]

Max tiap kolom: [1. 1. 1. 1. 1. 1. 1. 1. 1.]

Pembahasan. Array berukuran **(28.352, 9)**. Normalisasi dilakukan secara **vektorisasi penuh**:

`X.min(axis=0)` menghasilkan nilai minimum tiap kolom, dan berkat **broadcasting** NumPy, ekspresi `(X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))` diterapkan ke seluruh elemen sekaligus **tanpa satu pun loop Python**. Hasilnya, nilai minimum tiap kolom menjadi 0 dan maksimumnya 1.

Normalisasi ini penting karena fitur seperti `tempo` (~120) dan `loudness` (~-7) berskala jauh lebih besar daripada fitur lain yang berkisar 0-1. Tanpa penyetaraan skala, fitur berskala besar akan mendominasi perhitungan jarak/kemiripan pada tahap berikutnya.

Soal 10 — Correlation matrix & pasangan ekstrem

```
In [14]: C = np.corrcoef(X, rowvar=False)
corr_df = pd.DataFrame(C, index=feats, columns=feats)
display(corr_df.round(3))

mask = np.triu(np.ones_like(C, dtype=bool), k=1)
vals = np.where(mask, C, np.nan)

imax = np.unravel_index(np.nanargmax(vals), vals.shape)
imin = np.unravel_index(np.nanargmin(vals), vals.shape)

print("Paling Positif:", feats[imax[0]], "&", feats[imax[1]], round(C[imax[0], imax[1]], 3))
print("Paling Negatif:", feats[imin[0]], "&", feats[imin[1]], round(C[imin[0], imin[1]], 3))
```

	danceability	energy	loudness	speechiness	acousticness	instrumentalness	liveness
danceability	1.000	-0.081	0.015	0.184	-0.029	-0.002	-0.127
energy	-0.081	1.000	0.682	-0.029	-0.546	0.024	0.164
loudness	0.015	0.682	1.000	0.013	-0.372	-0.154	0.082
speechiness	0.184	-0.029	0.013	1.000	0.025	-0.108	0.059
acousticness	-0.029	-0.546	-0.372	0.025	1.000	-0.003	-0.075
instrumentalness	-0.002	0.024	-0.154	-0.108	-0.003	1.000	-0.008
liveness	-0.127	0.164	0.082	0.059	-0.075	-0.008	1.000
valence	0.334	0.150	0.050	0.065	-0.019	-0.174	-0.020
tempo	-0.185	0.152	0.097	0.033	-0.114	0.021	0.022

```
Paling Positif: energy & loudness 0.682
Paling Negatif: energy & acousticness -0.546
```

Jawaban & interpretasi musikal. Korelasi positif terkuat adalah **energy & loudness (+0,682)**, sementara yang paling negatif adalah **energy & acousticness (-0,546)**.

Catatan teknis: `rowvar=False` wajib digunakan agar tiap **kolom** diperlakukan sebagai satu variabel. `np.triu(..., k=1)` mengambil hanya segitiga atas matriks supaya diagonal (korelasi 1,0) dan pasangan kembar tidak ikut terpilih.

Secara musikal keduanya masuk akal. **Energy dan loudness** berkorelasi positif karena lagu yang lebih keras umumnya terdengar lebih energik — produksi yang padat menaikkan keduanya bersamaan. Sebaliknya, **energy dan acousticness** berkorelasi negatif: lagu akustik (gitar/piano polos, minim produksi elektronik) cenderung lebih tenang, sehingga ketika acousticness naik, energy turun.

Soal 11 — Boolean masking murni NumPy

Saya mengidentifikasi lagu yang `energy`-nya berada di atas $\mu + \sigma$ menggunakan *boolean masking* NumPy (bukan filtering Pandas), lalu membandingkan popularitas rata-ratanya dengan keseluruhan.

```
In [15]: energy = df_unique["energy"].values
popularity = df_unique["track_popularity"].values

threshold = energy.mean() + energy.std()
mask = energy > threshold

print("Threshold:", round(threshold, 3))
print("Jumlah lagu high-energy:", mask.sum())
print("Mean popularity (high-energy):", round(popularity[mask].mean(), 2))
print("Mean popularity (keseluruhan):", round(popularity.mean(), 2))
```

```
Threshold: 0.882
Jumlah lagu high-energy: 4852
Mean popularity (high-energy): 34.04
Mean popularity (keseluruhan): 39.34
```

Jawaban. Ada sekitar **4.852 lagu high-energy**. Yang menarik, **popularitas rata-ratanya justru lebih rendah** ($\approx 34,0$) dibanding mean popularitas keseluruhan ($\approx 39,3$).

Di sini `mask` adalah array boolean; `popularity[mask]` hanya mengambil elemen yang `True`, dan `mask.sum()` menghitung jumlah `True` (karena `True = 1`). Temuannya sedikit *counterintuitive*: lagu paling energik bukan yang paling populer. Artinya, intensitas audio yang tinggi **bukan resep otomatis** untuk *hits* — popularitas tampaknya lebih dipengaruhi faktor lain (artis, lirik, momentum rilis, pemasaran) daripada sekadar tingkat energi.

Stage 4 — Dokumentasi dan Refleksi AI Tool

Soal 12 — Docstring dengan bantuan AI tool

Pertama, saya membungkus dua analisis sebelumnya menjadi fungsi (normalisasi dari Soal 9 dan tabel ringkasan dari Soal 4), kemudian meminta bantuan AI coding tool untuk menyusun docstring-nya. Fungsi-fungsi ini juga menjadi fondasi untuk class di Bonus 3.

```
In [16]: def normalize_features(data: pd.DataFrame, feature_cols: list) -> np.ndarray:
        """Normalisasi fitur numerik ke rentang [0, 1] dengan min-max scaling.

        Args:
            data (pd.DataFrame): DataFrame yang memuat kolom-kolom fitur.
            feature_cols (list[str]): Daftar nama kolom yang akan dinormalisasi.

        Returns:
            np.ndarray: Array 2D berukuran (n_baris, n_fitur); setiap kolom
                berada pada rentang [0, 1].
        """
        X = data[feature_cols].values
        return (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))

def summarize_numeric(data: pd.DataFrame) -> pd.DataFrame:
    """Bangun tabel ringkasan statistik untuk seluruh kolom numerik.

    Menghitung mean, median, standar deviasi, dan IQR (Q3 - Q1). IQR dihitung
    manual menggunakan NumPy (np.nanpercentile), bukan scipy, sesuai instruksi.

    Args:
        data (pd.DataFrame): DataFrame sumber.

    Returns:
        pd.DataFrame: Tabel ringkasan berisi kolom mean, median, std, dan IQR,
            diindeks oleh nama kolom numerik dan dibulatkan ke 3 desimal.
    """
    cols = data.select_dtypes(include=np.number).columns
    summary = pd.DataFrame({
        "mean": data[cols].mean(),
        "median": data[cols].median(),
        "std": data[cols].std(),
    })
    q1 = np.nanpercentile(data[cols].values, 25, axis=0)
    q3 = np.nanpercentile(data[cols].values, 75, axis=0)
    summary["IQR"] = q3 - q1
    return summary.round(3)

# Uji singkat untuk memastikan fungsi berjalan
print("normalize_features ->", normalize_features(df_unique, feats).shape)
summarize_numeric(df).head()
```

normalize_features -> (28352, 9)

```
Out[16]:
```

	mean	median	std	IQR
track_popularity	42.484	45.000	24.980	38.000
danceability	0.655	0.672	0.145	0.198
energy	0.699	0.721	0.181	0.259
key	5.374	6.000	3.612	7.000
loudness	-6.720	-6.166	2.989	3.526

Refleksi penggunaan AI tool.

Prompt yang saya gunakan (untuk `normalize_features`):

"Buatkan docstring format Google style untuk fungsi Python berikut. Sertakan deskripsi singkat, bagian Args dengan tipe data tiap parameter, dan bagian Returns. [fungsi ditempel di sini]"

Output AI & evaluasi: AI menangkap tujuan fungsi dengan benar dan struktur Google style-nya rapi. Namun pada bagian `Returns`, output awalnya menulis tipe kembalian sebagai `pd.DataFrame`, padahal fungsi ini mengembalikan `np.ndarray`. Jadi saya **memodifikasi**, bukan memakai mentah-mentah: tipe return saya perbaiki menjadi `np.ndarray` dan saya tambahkan keterangan `shape (n_baris, n_fitur)` agar lebih informatif. Ini menegaskan poin handbook bahwa output AI **harus selalu diverifikasi**, terutama untuk detail teknis.

Prompt (untuk `summarize_numeric`): serupa, dengan tambahan konteks "jelaskan bahwa IQR dihitung manual".

Output AI & evaluasi: kali ini hasilnya cukup baik dan saya pakai **hampir apa adanya**, hanya menambahkan satu kalimat bahwa IQR dihitung manual dengan NumPy (bukan scipy), karena itu justru poin penting dari tugasnya yang tidak otomatis diketahui AI dari kode saja.

Catatan: docstring di atas adalah versi final setelah saya bandingkan dengan output AI dan saya sunting seperlunya — bukan menyalin buta.

Soal 13 — Tiga insight untuk pembaca non-teknis

Berikut tiga temuan utama dari analisis ini, ditulis agar mudah dimengerti orang awam:

- 1. Banyak lagu belum tentu lagu bagus.** Queen punya lagu terbanyak di dataset (130), tetapi rata-rata popularitasnya kalah dari David Guetta yang lagunya lebih sedikit. Artinya, produktif belum tentu berarti tiap karyanya disukai banyak orang.
- 2. Lagu yang lebih "kencang dan energik" justru cenderung kurang populer.** Lagu dengan energi paling tinggi punya popularitas rata-rata di bawah lagu pada umumnya. Jadi membuat lagu yang menghentak saja bukan jaminan untuk menjadi hits.
- 3. Genre R&B paling sulit ditebak selera pasarnya.** Popularitas lagu R&B sangat naik-turun — ada yang sangat terkenal, banyak juga yang sepi — berbeda dengan EDM yang lebih seragam. Bagi aplikasi musik, ini berarti rekomendasi lagu R&B perlu lebih dipersonalisasi untuk tiap pengguna.

Bonus 1 — Artist Audio Fingerprint & Cosine Similarity

Saya membuat fungsi `artist_fingerprint` yang mengembalikan rata-rata sembilan fitur audio seorang artis, lalu membandingkan dua artis dengan cosine similarity yang diimplementasikan dari definisi matematisnya.

Catatan penting: sebelum membandingkan, fitur **harus dinormalisasi**. Jika memakai fitur mentah, `tempo` (~120) dan `loudness` akan mendominasi sehingga semua artis tampak nyaris identik (~1,0) dan hasilnya tidak informatif. Maka saya menghitung *fingerprint* di atas data yang sudah dinormalisasi.

```
In [17]: X = df_unique[feats].values
X_norm = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
df_norm = df_unique.copy()
df_norm[feats] = X_norm

def artist_fingerprint(data, artist_name):
    """Return mean vector 9 fitur audio seorang artis sebagai NumPy array."""
    return data[data["track_artist"] == artist_name][feats].mean().values

def cosine_similarity(A, B):
    """Cosine similarity dari definisi: A·B / (||A|| * ||B||)."""
    return np.dot(A, B) / (np.linalg.norm(A) * np.linalg.norm(B))

# Pengetesan minimal tiga pasang artis (sesuai instruksi)
pairs = [
    ("Martin Garrix", "David Guetta"), # sama-sama DJ EDM
    ("Queen", "Guns N' Roses"), # sama-sama rock
    ("David Guetta", "Queen"), # beda genre (EDM vs rock)
```

```

]
for a, b in pairs:
    sim = cosine_similarity(artist_fingerprint(df_norm, a),
                           artist_fingerprint(df_norm, b))
    print(f"{a} vs {b}: {sim:.4f}")

```

Martin Garrix vs David Guetta: 0.9849

Queen vs Guns N' Roses: 0.9685

David Guetta vs Queen: 0.9803

Interpretasi. Hasilnya: Garrix-Guetta $\approx 0,985$, Queen-GnR $\approx 0,969$, dan Guetta-Queen $\approx 0,980$.

Pasangan sesama DJ EDM (Garrix-Guetta) memang yang **paling mirip** — sesuai intuisi. Namun hipotesis "artis satu genre **selalu** lebih mirip" **tidak sepenuhnya terbukti**: pasangan beda-genre (Guetta-Queen, 0,980) malah skornya sedikit di atas pasangan sesama rock (Queen-GnR, 0,969). Penyebabnya, pada vektor yang semua elemennya positif (rentang [0,1]), nilai cosine cenderung tinggi dan rapat (0,96–0,99), sehingga yang lebih bermakna adalah **peringkat relatifnya**, bukan angka absolutnya. Untuk kontras yang lebih tajam, membandingkan artis akustik melawan artis EDM akan memberi selisih yang lebih jelas.

Bonus 2 — Genre Cluster Profile

Saya membangun "profil audio" tiap genre lewat **centroid** (rata-rata vektor fitur per genre), lalu menghitung **matriks jarak Euclidean** antar-genre secara vektorisasi penuh dengan *broadcasting*. Karena ini analisis genre, saya memakai **df** penuh, dan tetap menormalisasi fitur agar setara.

```

In [18]: Xg = df[feats].values
Xg_norm = (Xg - Xg.min(axis=0)) / (Xg.max(axis=0) - Xg.min(axis=0))
df_gnorm = df.copy()
df_gnorm[feats] = Xg_norm

genres = sorted(df_gnorm["playlist_genre"].unique())

# 1. Centroid tiap genre -> matriks (n_genres, 9)
centroids = np.array([df_gnorm[df_gnorm["playlist_genre"] == g][feats].mean().values
                      for g in genres])
print("Shape centroid:", centroids.shape)  # (6, 9)

# 2. Pairwise Euclidean distance pakai BROADCASTING (tanpa loop)
diff = centroids[:, None, :] - centroids[None, :, :]  # (6,1,9) - (1,6,9) = (6,6,9)
dist = np.sqrt((diff ** 2).sum(axis=2))  # (6,6)
dist_df = pd.DataFrame(dist.round(3), index=genres, columns=genres)
display(dist_df)

# 3. Genre paling mirip & paling beda
mask = np.triu(np.ones_like(dist, dtype=bool), k=1)
vals = np.where(mask, dist, np.nan)
imin = np.unravel_index(np.nanargmin(vals), vals.shape)
imax = np.unravel_index(np.nanargmax(vals), vals.shape)
print("Paling MIRIP:", genres[imin[0]], "&", genres[imin[1]], round(dist[imin], 3))
print("Paling BEDA :", genres[imax[0]], "&", genres[imax[1]], round(dist[imax], 3))

```

Shape centroid: (6, 9)

	edm	latin	pop	r&b	rap	rock
edm	0.000	0.324	0.239	0.372	0.297	0.273
latin	0.324	0.000	0.139	0.160	0.162	0.230
pop	0.239	0.139	0.000	0.164	0.168	0.140
r&b	0.372	0.160	0.164	0.000	0.150	0.255
rap	0.297	0.162	0.168	0.150	0.000	0.273
rock	0.273	0.230	0.140	0.255	0.273	0.000

Paling MIRIP: latin & pop 0.139

Paling BEDA : edm & r&b 0.372

Cara kerja broadcasting (bagian tersulit). `centroids[:, None, :]` mengubah bentuk dari (6, 9) menjadi (6, 1, 9), dan `centroids[None, :, :]` menjadi (1, 6, 9). Saat dikurangkan, NumPy "meregangkan" keduanya menjadi (6, 6, 9) — tiap sel `[i, j]` berisi selisih sembilan fitur antara genre *i* dan *j*. Inilah yang memungkinkan seluruh pasangan dihitung sekaligus tanpa loop.

Hasil & kesesuaian dengan intuisi. Genre **paling mirip: latin & pop (0,139); paling berbeda: edm & r&b (0,372)**. Keduanya cocok dengan intuisi musik: latin dan pop sama-sama bercorak *mainstream* dan *danceable*, sedangkan edm (elektronik, energi tinggi) berseberangan dengan r&b yang lebih *smooth* dan tenang.

```
In [19]: def recommend_similar_genre(genre, top_k=3):
        idx = genres.index(genre)
        distances = dist[idx]
        order = np.argsort(distances)          # urut dari jarak terkecil
        order = order[order != idx]            # buang dirinya sendiri
        return [genres[i] for i in order[:top_k]]

print("Mirip dengan rock :", recommend_similar_genre("rock"))
print("Mirip dengan edm  :", recommend_similar_genre("edm"))
```

```
Mirip dengan rock : ['pop', 'latin', 'r&b']
Mirip dengan edm  : ['pop', 'rock', 'rap']
```

Fungsi `recommend_similar_genre` memanfaatkan matriks jarak: `np.argsort` mengembalikan urutan indeks dari jarak terkecil ke terbesar (jarak terkecil = paling mirip), lalu genre itu sendiri dibuang dari hasil sebelum diambil `top_k` teratas.

Bonus 3 — Reusable Analysis Class: SpotifyAnalyzer

Seluruh analisis saya rangkum ke dalam satu class dengan lebih dari lima method, **type hints lengkap** pada setiap method, dan sebuah `generate_report()` yang mengembalikan dictionary siap dicetak atau diserialisasi ke JSON. Class ini dirancang agar berjalan dari **satu cell** tanpa setup tambahan.

```
In [20]: import pandas as pd
import numpy as np
import json

class SpotifyAnalyzer:
    """Pipeline EDA dataset lagu Spotify: load, analisis, dan ringkas insight.

    Attributes:
        df (pd.DataFrame): Data lengkap (setelah baris tanpa identitas dibuang),
            dipakai untuk analisis berbasis genre.
        df_unique (pd.DataFrame): Data unik per track_id, dipakai untuk analisis
            fitur audio per-lagu agar tidak bias oleh duplikat antar-playlist.
    """

    FEATURES = ["danceability", "energy", "loudness", "speechiness",
                "acousticness", "instrumentalness", "liveness", "valence", "tempo"]

    def __init__(self, url: str) -> None:
        """Muat dataset dari URL lalu siapkan dua versi data (df dan df_unique).

        Args:
            url (str): URL CSV dataset Spotify.
        """
        self.df: pd.DataFrame = pd.read_csv(url)
        self.df = self.df.dropna(subset=["track_name", "track_artist"]).reset_index(drop=True)
        self.df_unique: pd.DataFrame = self.df.drop_duplicates(
            subset="track_id", keep="first").reset_index(drop=True)

    def summarize_numeric(self) -> pd.DataFrame:
        """Ringkasan mean, median, std, dan IQR (manual) tiap kolom numerik.

        Returns:
            pd.DataFrame: Tabel statistik per kolom numerik.
        """
        cols = self.df.select_dtypes(include=np.number).columns
        summary = pd.DataFrame({
            "mean": self.df[cols].mean(),
            "median": self.df[cols].median(),
            "std": self.df[cols].std(),
        })
        q1 = np.nanpercentile(self.df[cols].values, 25, axis=0)
        q3 = np.nanpercentile(self.df[cols].values, 75, axis=0)
        summary["IQR"] = q3 - q1
        return summary.round(3)
```

```

def popularity_variance_by_genre(self) -> pd.Series:
    """Variance track_popularity per genre, terurut menurun.

    Returns:
        pd.Series: Variance popularitas tiap genre.
    """
    return self.df.groupby("playlist_genre")["track_popularity"].var().sort_values(ascending=False)

def top_artists(self, n: int = 10) -> pd.DataFrame:
    """n artis dengan lagu unik terbanyak beserta mean popularitasnya.

    Args:
        n (int): Jumlah artis teratas. Default 10.

    Returns:
        pd.DataFrame: Kolom n_tracks dan mean_popularity per artis.
    """
    top = self.df_unique["track_artist"].value_counts().head(n)
    pop = (self.df_unique[self.df_unique["track_artist"].isin(top.index)]
           .groupby("track_artist")["track_popularity"].mean())
    return pd.DataFrame({"n_tracks": top, "mean_popularity": pop.round(2)})

def correlation_pairs(self) -> dict:
    """Pasangan fitur audio dengan korelasi paling positif dan paling negatif.

    Returns:
        dict: Kunci 'most_positive' dan 'most_negative', masing-masing berisi
              (fitur_1, fitur_2, nilai_korelasi).
    """
    X = self.df_unique[self.FEATURES].values
    C = np.corrcoef(X, rowvar=False)
    mask = np.triu(np.ones_like(C, dtype=bool), k=1)
    vals = np.where(mask, C, np.nan)
    imax = np.unravel_index(np.nanargmax(vals), vals.shape)
    imin = np.unravel_index(np.nanargmin(vals), vals.shape)
    return {
        "most_positive": (self.FEATURES[imax[0]], self.FEATURES[imax[1]], round(float(C[imax]), 3),
        "most_negative": (self.FEATURES[imin[0]], self.FEATURES[imin[1]], round(float(C[imin]), 3)
    }

def high_energy_comparison(self) -> dict:
    """Bandingkan popularitas lagu high-energy (>mu+sigma) vs keseluruhan.

    Returns:
        dict: Jumlah lagu high-energy dan dua nilai mean popularitas.
    """
    energy = self.df_unique["energy"].values
    pop = self.df_unique["track_popularity"].values
    mask = energy > (energy.mean() + energy.std())
    return {
        "n_high_energy": int(mask.sum()),
        "mean_pop_high_energy": round(float(pop[mask].mean()), 2),
        "mean_pop_overall": round(float(pop.mean()), 2),
    }

def generate_report(self) -> dict:
    """Rangkum seluruh insight utama ke dalam dictionary siap-serialisasi JSON.

    Returns:
        dict: Ringkasan temuan kunci dari seluruh tahap analisis.
    """
    var = self.popularity_variance_by_genre()
    arts = self.top_artists()
    corr = self.correlation_pairs()
    energy = self.high_energy_comparison()
    return {
        "dataset_shape": list(self.df.shape),
        "unique_tracks": int(self.df_unique.shape[0]),
        "highest_variance_genre": str(var.index[0]),
        "most_productive_artist": str(arts["n_tracks"].idxmax()),
        "most_popular_top_artist": str(arts["mean_popularity"].idxmax()),
        "correlation": corr,
        "energy_vs_popularity": energy,
    }

```

```
# Demonstrasi: jalankan dan tampilkan laporan
url = "https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2020/2020-01-21/spotify_tracks.csv"
analyzer = SpotifyAnalyzer(url)
report = analyzer.generate_report()
print(json.dumps(report, indent=2))

{
  "dataset_shape": [
    32828,
    23
  ],
  "unique_tracks": 28352,
  "highest_variance_genre": "r&b",
  "most_productive_artist": "Queen",
  "most_popular_top_artist": "David Guetta",
  "correlation": {
    "most_positive": [
      "energy",
      "loudness",
      0.682
    ],
    "most_negative": [
      "energy",
      "acousticness",
      -0.546
    ]
  },
  "energy_vs_popularity": {
    "n_high_energy": 4852,
    "mean_pop_high_energy": 34.04,
    "mean_pop_overall": 39.34
  }
}
```

Catatan desain class & refleksi AI.

- **FEATURES** sebagai **class attribute** agar tidak ditulis berulang di banyak method (prinsip *DRY*).
- **Type hints lengkap** pada setiap method (mis. `url: str`, `n: int = 10`, `-> pd.DataFrame`) untuk memperjelas kontrak input/output, sesuai permintaan tugas.
- **Konversi `int()` / `float()`** pada `generate_report` bersifat wajib: hasil NumPy (`np.int64` / `np.float64`) tidak bisa langsung diserialisasi oleh `json.dumps`, jadi diubah ke tipe Python asli terlebih dahulu.
- **`__init__` langsung menyiapkan `df` dan `df_unique`**, konsisten dengan keputusan deduplikasi di Stage 1.

Refleksi AI tool untuk docstring method: prompt yang saya pakai serupa Soal 12, yaitu meminta docstring Google style untuk tiap method beserta Args/Returns. Output AI umumnya benar untuk method sederhana, tetapi untuk `generate_report` AI tidak menyebut alasan konversi tipe NumPy ke Python — bagian itu saya tambahkan sendiri karena AI tidak bisa menyimpulkannya hanya dari kode. Sekali lagi, AI berguna sebagai *co-pilot* untuk draf awal, namun verifikasi dan penyuntingan tetap tanggung jawab saya.

Notebook ini telah dirancang agar berjalan berurutan dari atas ke bawah (Runtime → Restart and run all) tanpa error.